Two Pointers

> → **Key Idea**: Use two pointers moving towards each other or in same direction to solve array problems efficiently

> • **When to use**: Sorted arrays, finding pairs, removing duplicates, palindromes, sliding patterns

> ⊕ **Complexity**: Time O(n), Space O(1)

> ⚠ **Common Pitfall**: Not handling edge cases like empty arrays or single elements

Overview

Visual Example

Two Pointers Technique

```
Target Sum Problem: Find pair that sums to 9
Array: [1, 3, 4, 6, 8, 9, 11]

Step 1: left=0, right=6
[1, 3, 4, 6, 8, 9, 11]    1 + 11 = 12 > 9, move right pointer left
 ↑                 ↑

Step 2: left=0, right=5
[1, 3, 4, 6, 8, 9, 11]    1 + 9 = 10 > 9, move right pointer left
 ↑              ↑

Step 3: left=0, right=4
[1, 3, 4, 6, 8, 9, 11]    1 + 8 = 9 ✓ FOUND!
 ↑           ↑

🗲 O(n) solution vs O(n²) brute force
```

Fast & Slow Pointers (Cycle Detection)

```
Linked List with Cycle:
1 → 2 → 3 → 4 → 5
    ↑           ↓
    8 ← 7 ← 6 ←

Step 1: slow=1, fast=2     Step 2: slow=2, fast=4     Step 3: slow=3, fast=6
Step 4: slow=4, fast=8     Step 5: slow=5, fast=3     Step 6: slow=6, fast=7
```

```
Step 7: slow=7, fast=5     Step 8: slow=8, fast=2     Step 9: slow=2, fast=6

>> When slow == fast, cycle detected!
```

The two-pointer technique uses two pointers that traverse the data structure in a coordinated way. The pointers can move toward each other (opposite direction) or in the same direction at different speeds.

This technique is particularly powerful because it often reduces time complexity from $O(n^2)$ to $O(n)$ while maintaining $O(1)$ space complexity.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Pattern	Time Complexity	Space Complexity
Two Sum (sorted)	$O(n)$	$O(1)$
Three Sum	$O(n^2)$	$O(1)$
Cycle Detection	$O(n)$	$O(1)$
Array Partitioning	$O(n)$	$O(1)$

Container Water	O(n)	O(1)

The two-pointer technique typically achieves linear time complexity with constant space, making it highly efficient for interview problems.

Common Problem Patterns

1. Two Sum II (Sorted Array)

Problem: Given a sorted array, find two numbers that add up to a target.

```
public int[] twoSum(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return new int[]{left + 1, right + 1};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

2. Three Sum

Problem: Find all unique triplets that sum to zero.

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue; // Skip duplicates

        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left+1]) left++;
                while (left < right && nums[right] == nums[right-1]) right--;
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
```

```
        return result;
    }
```

3. Container With Most Water

Problem: Find two lines that together with x-axis forms a container with most water.

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxWater = 0;

    while (left < right) {
        int water = Math.min(height[left], height[right]) * (right - left);
        maxWater = Math.max(maxWater, water);

        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}
```

4. Remove Duplicates from Sorted Array

Problem: Remove duplicates in-place, return new length.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

5. Linked List Cycle Detection

Problem: Detect if a linked list has a cycle.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
```

```
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
```

← Previous: Binary Search | Back to Main Guide | Next: Sliding Window →